

Quantum-Safe Tactical Communication System - Project Report

PART 0: What We're Actually Building (The Big Picture)

Imagine you're a defence unit (platoon/drone squad) in the field. You need to send secret messages (position, orders, intel) to HQ over untrusted networks (4G, radio, internet). Today's encryption can be recorded and broken later by quantum computers.

Your mission: Build a communication system that:

- Generates unbreakable keys using quantum mechanics (via Qiskit simulator)
- Encrypts messages with those quantum keys
- Detects attacks automatically if someone tries to eavesdrop
- Works end-to-end: soldier device -> gateway -> HQ with monitoring

Think of it like: BB84 QKD (quantum) + secure messaging (classical) + attack detection (monitoring) = a complete defence communication prototype.

PART 1: Quantum Basics You Need to Know

What is BB84? (The quantum protocol we'll use)

BB84 is a quantum key distribution protocol invented in 1984. Here's how it works:

Scene: Alice (HQ) and Bob (soldier in field) want to share a secret key. Eve (hacker) is trying to intercept.

The protocol:

1. Alice generates random bits (0s and 1s) and random bases.
 - Basis 1 (computational): measure in standard 0/1 state
 - Basis 2 (Hadamard): measure in diagonal +/- state
2. For each bit, Alice encodes it in a qubit using her chosen basis:
 - Bit 0, basis 1 -> $|0\rangle$
 - Bit 1, basis 1 -> $|1\rangle$
 - Bit 0, basis 2 -> $|+\rangle$
 - Bit 1, basis 2 -> $|-\rangle$
3. Alice sends the qubits to Bob.
4. Bob randomly chooses bases to measure each qubit.
 - If Bob's basis matches Alice's, result matches Alice's bit.
 - If Bob's basis is wrong, result is 50/50 random.
5. Sifting step: Alice and Bob compare bases (not bits). Keep only bits where bases matched.

Attack detection:

- Eve sees qubits but doesn't know bases.
- Eve guesses a basis and measures (50/50 chance of being right).
- When Eve measures wrong, the qubit's state collapses.
- Result: Bob's subsequent measurement will likely differ from Alice's even if bases matched.
- Error rate (QBER) spikes -> Attack detected!

Quantum-Safe Tactical Communication System - Project Report

In your system: Qiskit simulates this protocol, produces a shared key, and calculates QBER. If $QBER > 11\%$, we know there's an eavesdropper.

PART 2: System Architecture

1. Soldier Devices (A/B): Send encrypted messages. Request fresh encryption keys from KMS.
2. Gateway: Routes encrypted messages between field devices and HQ.
3. KMS (Key Management Service): Central service that receives key requests, calls Qiskit simulator, and distributes keys. Monitors QBER.
4. Qiskit BB84 Simulator: Simulates protocol, generates raw key, calculates error rate.
5. Monitoring Dashboard: Web interface showing system status and attack alerts.

PART 3: Step-by-Step Message Journey

Step 1: Soldier A requests a fresh key locally.

Step 2: KMS runs Qiskit BB84 simulator.

- Simulates Alice/Bob interaction.
- Calculates QBER.

Step 3: KMS checks for attacks.

- If $QBER > 11\%$, link is flagged RED. No key issued.
- If $QBER < 11\%$, key is safe.

Step 4: KMS derives session key (AES-256).

Step 5: Soldier A encrypts message using AES-GCM and sends packet.

Step 6: Soldier B decrypts message.

- Soldier B requests key from KMS (Verification/Demo logic ensures matched key).
- Decrypts ciphertext.

Step 7: Dashboard updates status (Green/Red, Key Count, etc.).

PART 4: Project Code Structure

```
quantum-tactical-comms/  
|-- quantum_engine/  
| |-- bb84_simulator.py (The Quantum Part)  
|-- kms/  
| |-- key_management_service.py (The Server)  
|-- devices/  
| |-- client.py (The Client)  
|-- gateway/  
| |-- network_gateway.py (The Router)  
|-- dashboard/  
| |-- dashboard_ui.py (The UI)  
|-- main.py (The Demo)
```

Quantum-Safe Tactical Communication System - Project Report

PART 5: For Your iDEX Proposal

Title: Quantum-Ready Tactical Communication System (QTCS)

Problem: Defence units need unhackable communication that works against future quantum computers.

Solution:

- Uses real BB84 quantum protocol (simulated via Qiskit).
- Generates quantum-safe keys for encryption.
- Automatically detects eavesdropping.
- Designed to plug into real QKD networks.

Key Innovation:

- Edge software for quantum networks (not just backbone).
- Software-defined, adaptable to any quantum source.

Demo: Shows secure platoon comms working end-to-end with attack detection.

PART 6: Key Technologies

- **BB84**: Quantum key distribution protocol.
- **Qiskit**: IBM's quantum computing SDK (simulates qubits).
- **QBER**: Quantum Bit Error Rate (attack metric).
- **AES-GCM**: Advanced Encryption Standard (symmetric encryption).
- **Streamlit**: Dashboard framework.